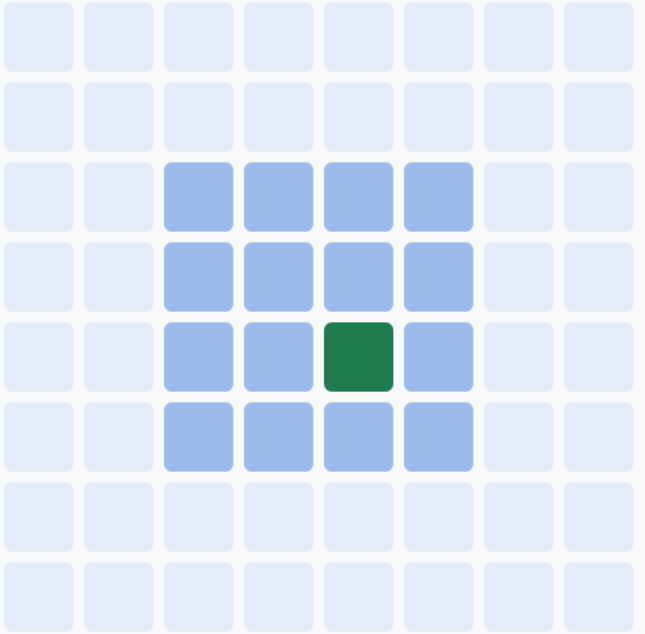
SEMINARVORTRAG · HOCHSCHULE KARLSRUHE (HKA)

Von C++ zu SYCL

Matrixmultiplikation in vier Stufen

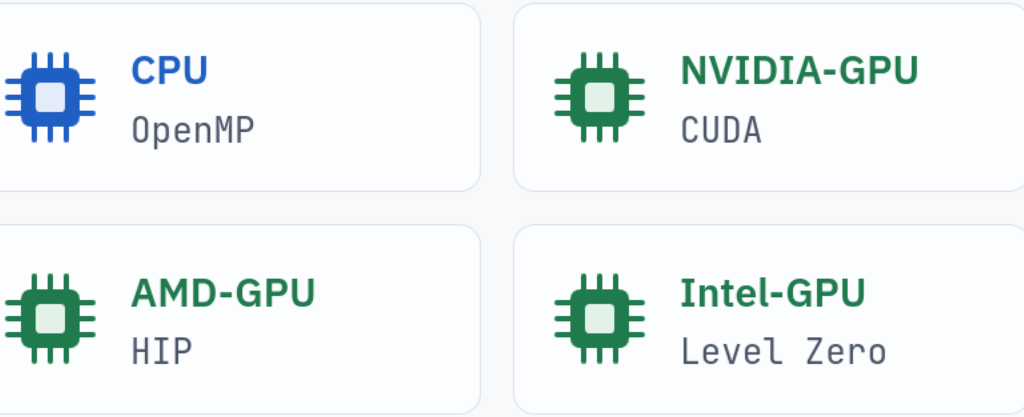
Portabler Code für CPU und GPU

Erlind Sejdiu · [DATUM]



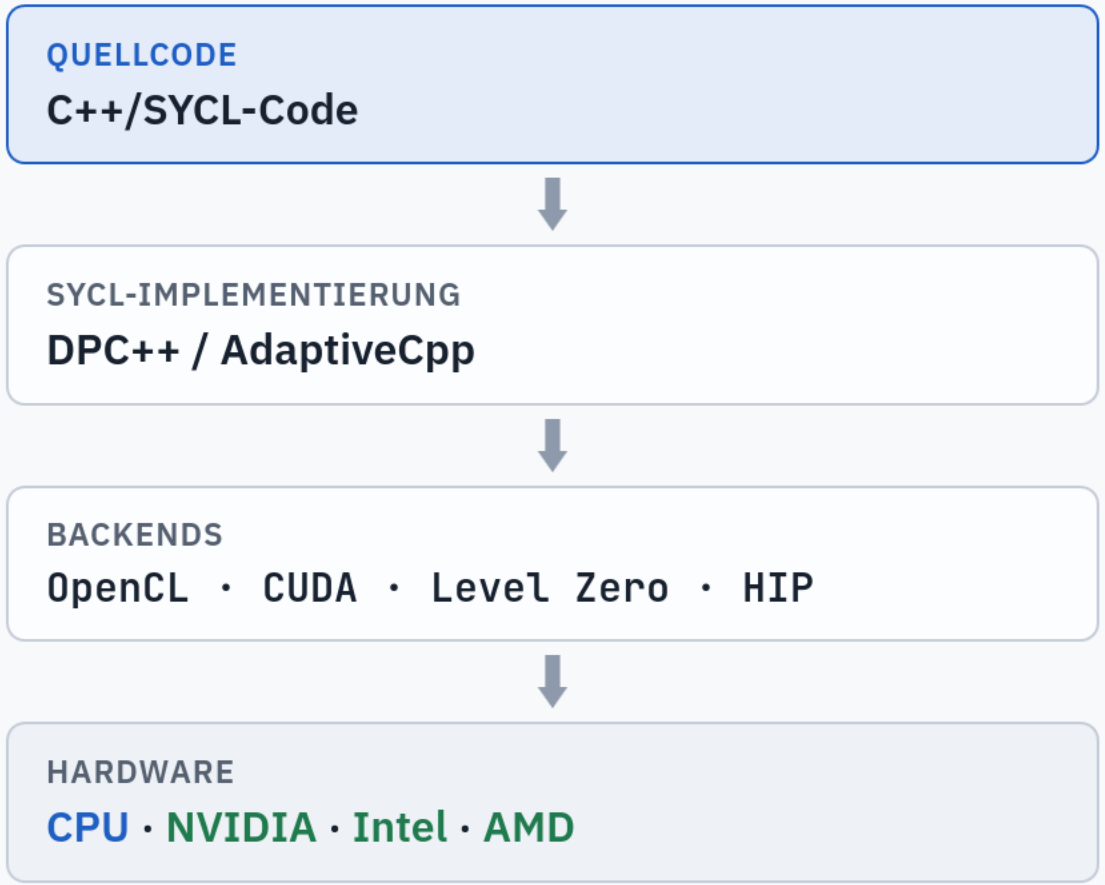
Motivation

- Moderne Rechner sind heterogen: CPU, NVIDIA-, AMD-, Intel-GPU
- Jeder Hersteller hat eine eigene Programmierumgebung (CUDA, HIP, ...)
- **Problem:** Code wird an einen Hersteller gebunden
- **Frage:** Geht es mit einem Quellcode für alle?



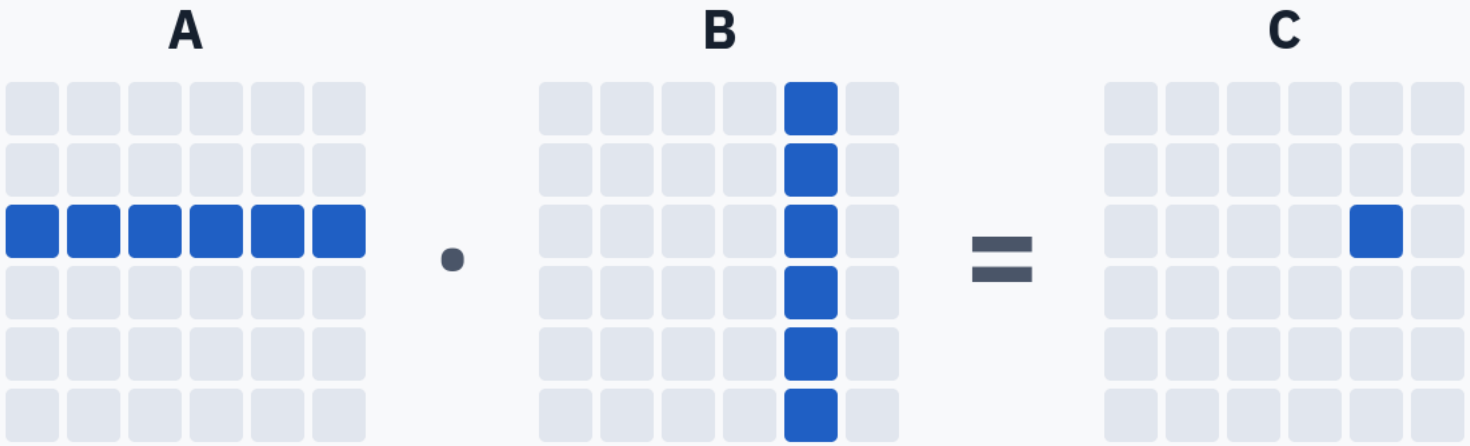
Was ist SYCL?

- Offener Standard der Khronos Group (wie OpenGL/ Vulkan)
- Reines C++17, kein Spracherweiterungs-Dialekt
- **Ein Quellcode → viele Backends**
- Implementierungen: Intel DPC++ (oneAPI), AdaptiveCpp



Beispielproblem: Matrixmultiplikation

- **C = A · B**, Matrizen n × n (float)
- Jedes Element C[i][j] = Skalarprodukt Zeile i von A × Spalte j von B
- Aufwand O(n³), alle C[i][j] unabhängig → **ideal parallelisierbar**



Vorgehen und Messmethodik

Stufe	Was ändert sich?	Neues SYCL-Konzept
0	Ausgangsprogramm, 3 Schleifen, 1 CPU-Kern	–
1	Schleifen → parallel_for, gemeinsamer Speicher	Queue, Kernel, range/id, USM shared
2	Speicher getrennt, Kopieren explizit	malloc_device, memcpy
3	Kacheln im schnellen Local Memory	nd_range, Work-Group, local_accessor, Barriere
4	Kein neuer Code – nur anderes Gerät	Geräteauswahl, Backends

Gemessen: Median aus 5 Läufen · erster Lauf separat · Ergebnis wird gegen double-Referenz geprüft · n = 256 bis 2048

Stufe 0 – Sequentiell

AUSGANGSPROGRAMM · C++

```
for (size_t i = 0; i < n; ++i)
  for (size_t j = 0; j < n; ++j) {
    float sum = 0.0f;
    for (size_t k = 0; k < n; ++k)
      sum += A[i*n + k] * B[k*n + j];
    C[i*n + j] = sum;
  }
```

n = 1024

4,5 s

0,48 GFLOPS

1 CPU-Kern

Stufe 1 – parallel_for + USM shared

VORHER · STUFE 0

```
- for (size_t i = 0; i < n; ++i)
-   for (size_t j = 0; j < n; ++j) {
```

NACHHER · STUFE 1

```
+ sycl::queue q;
+ float* A = sycl::malloc_shared<float>(n*n, q);
+ q.parallel_for(sycl::range<2>{n, n}, [=](sycl::id<2> idx) {
+   size_t i = idx[0], j = idx[1];
    float sum = 0.0f;
    for (size_t k = 0; k < n; ++k)
      sum += A[i*n + k] * B[k*n + j];
    C[i*n + j] = sum;
+ }).wait();
```

NEUE KONZEPTE

queue
Verbindung zum Gerät

Kernel
Lambda, läuft für jedes (i, j) parallel

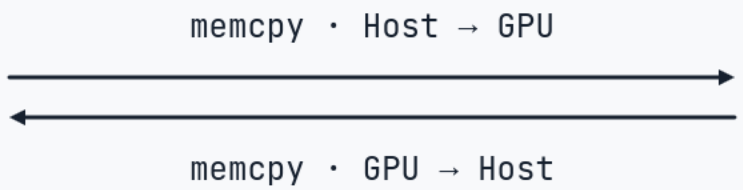
malloc_shared
Speicher für CPU und GPU sichtbar, wird automatisch verschoben

Stufe 2 – Expliziter Gerätespeicher

```
- float* A = sycl::malloc_shared<float>(n*n, q);
+ float* A = sycl::malloc_device<float>(n*n, q);
+ q.memcpy(A, hA.data(), bytes);   // Host → GPU
  // ... gleicher Kernel wie Stufe 1 ...
+ q.memcpy(hC.data(), C, bytes);   // GPU → Host
```

- Kernel unverändert
- Daten liegen nur auf der GPU
- Transfer wird **sichtbar und messbar**

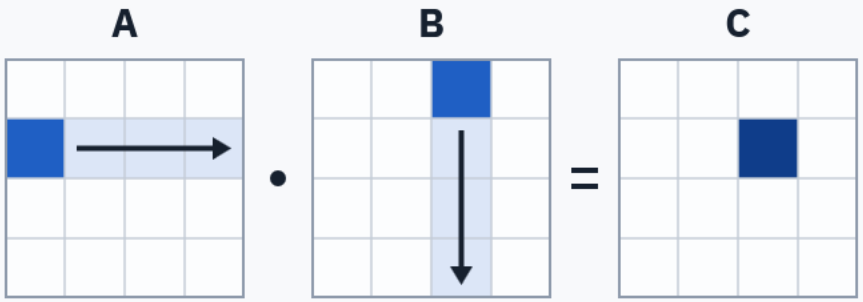
Host-RAM
hA, hB, hC
CPU-Seite



GPU-Speicher
A, B, C
Kernel rechnet hier

Stufe 3 – Tiling mit Local Memory

1. Work-Items bilden Work-Groups von 16 × 16
2. Gruppe lädt Kachel von A und B in Local Memory
3. **Barriere:** warten, bis alle geladen haben
4. Mit Kacheldaten rechnen, dann nächste Kachel



Kachel in A wandert nach rechts, Kachel in B nach unten – beide liefern die Ergebnis-Kachel in C

```
+ sycl::local_accessor<float,2> tileA({T,T}, h);
+ h.parallel_for(sycl::nd_range<2>{{n,n},{T,T}},
+     [=](sycl::nd_item<2> it) {
    tileA[li][lj] = A[i*n + t + lj];
+   sycl::group_barrier(it.get_group());
    for (k = 0; k < T; ++k) sum += tileA[li][k] * tileB[k][lj];
  });
```

Effekt: Jeder Wert wird pro Gruppe nur einmal statt 16-mal aus dem langsamen globalen Speicher gelesen.

Stufe 4 – Portabilität

Kein neuer Code

Gerät wird beim Start gewählt:

```
$ ONEAPI_DEVICE_SELECTOR=opencl:cpu ./stufe3_tiling
$ ONEAPI_DEVICE_SELECTOR=cuda:gpu   ./stufe3_tiling
```

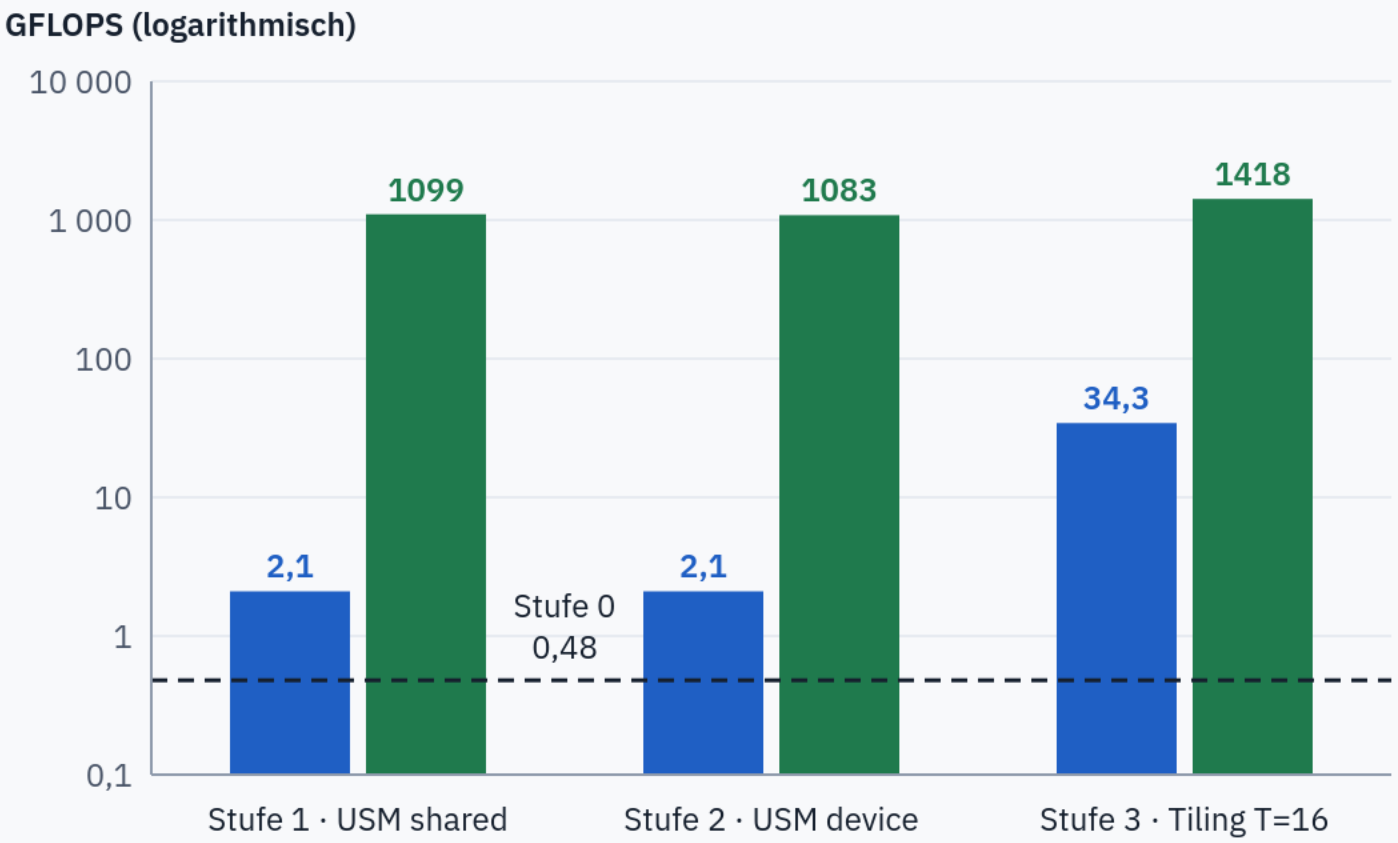
Ausgabe von sycl-ls:

```
$ sycl-ls
[opencl:cpu] AMD Ryzen 5 5600X 6-Core Processor
[cuda:gpu]   NVIDIA GeForce RTX 5060 Ti [CUDA 13.4]
```

Gleiche Binärdatei-Quelle, anderes Gerät.

Stufe 3 läuft unverändert auf CPU und GPU

Ergebnisse: Leistung je Stufe (n = 2048)



CPU · AdaptiveCpp, OpenMP, 6 Kerne

GPU · RTX 5060 Ti, DPC++

Stufe 0 · 1 CPU-Kern, n = 1024

Effekt von Tiling

+30 % auf GPU

×16 auf CPU

GPU vs. Stufe 0 · n = 1024

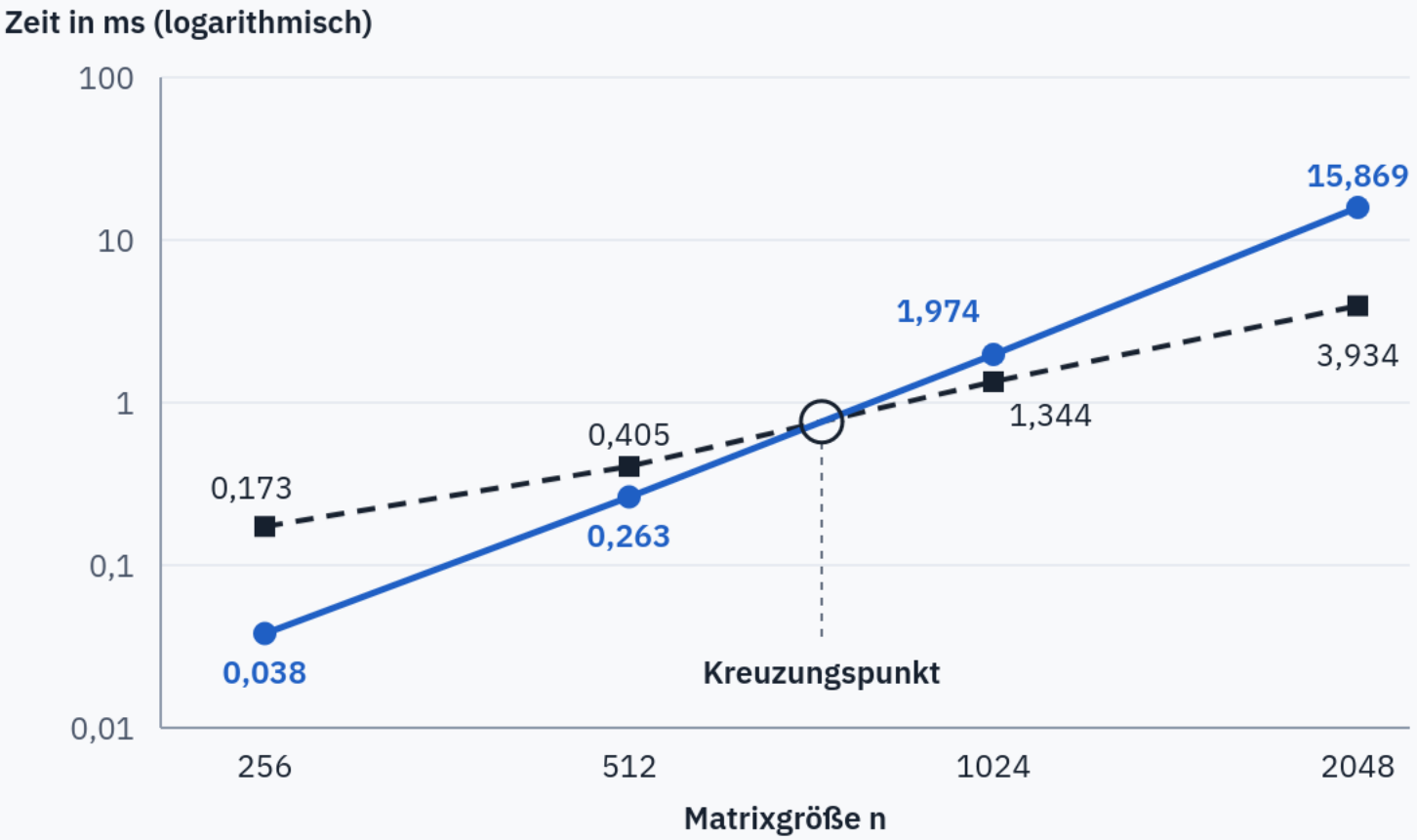
~3000× schneller

`4484 ms → 1,5 ms`

⚠ **nicht fair (1 Kern vs. GPU)**

CPU: AdaptiveCpp, OpenMP-Backend · GPU: DPC++ v7.2.0 (intel/llvm), CUDA 13.4 · CPU: AMD Ryzen 5 5600X

Ergebnisse: Transfer vs. Rechnung



GPU · STUFE 2

Rechnung (Kernel, Median)

Transfer hin + zurück

Bei kleinen Matrizen kostet das Kopieren mehr als das Rechnen – die GPU lohnt sich erst ab ~n = 1000.

Schwierigkeiten

Erster Lauf deutlich langsamer

JIT-Kompilierung, erster Transfer

n = 1024, Stufe 1: **4,3 ms** vs. **1,9 ms** im Median

Toolchain-Setup aufwendiger als der Code

CUDA-Backend fehlte in Intel oneAPI → Open-Source-DPC++ verwendet

Beste Kachelgröße ist geräteabhängig

Portabilität ≠ Performance-Portabilität

Stufe 1 ≈ Stufe 2 im Median

Shared Memory verschiebt Daten nur beim ersten Zugriff

Fazit

1

Der Umstieg ist schrittweise möglich

wenige Zeilen pro Stufe

2

Ein Quellcode läuft auf CPU und GPU

Gerät wird erst beim Start gewählt

3

Für Spitzenleistung braucht es hardwarenahes Tuning

Tiling, Kachelgröße

Ausblick AMD-/Intel-GPU messen · Kachelgrößen 8/16/32 vergleichen · Vergleich mit CUDA/cuBLAS

VIELEN DANK

Fragen?

Erlind Sejdiu · [E-Mail]

Code und Messdaten: [Repository-Link]